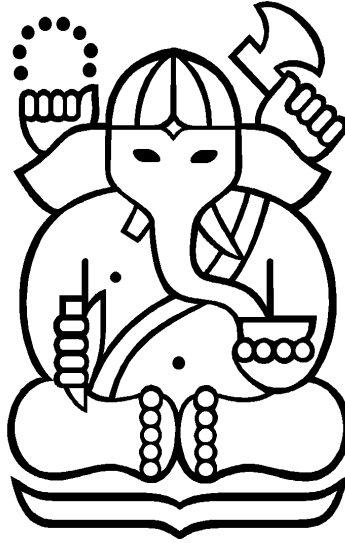


LAPORAN TUGAS BESAR 1
IF3270 PEMBELAJARAN MESIN
FEED FORWARD NEURAL NETWORK (FFNN)
SEMESTER II TAHUN 2025/2026



DISUSUN OLEH :

MUHAMMAD RAIHAN NAZHIM OKTANA (13523021)

MUHAMMAD LUQMAN HAKIM (13523044)

FAQIH MUHAMMAD SYUHADA (13523057)

Dosen Pengampu :

Dr. Fariska Zakhralativa Ruskanda, S.T., M.T.

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

LAPORAN TUGAS BESAR 1.....	1
DAFTAR ISI.....	2
I. Deskripsi Persoalan.....	3
II. Pembahasan.....	8
2.1 Penjelasan Implementasi.....	8
2.1.1 Implementasi Kelas.....	8
2.1.2 Implementasi Forward Propagation.....	8
2.1.3 Implementasi Backward Propagation.....	9
2.2 Hasil Pengujian.....	9
2.2.1 Pengaruh Depth dan Width.....	9
2.2.2 Pengaruh Fungsi Aktivasi.....	12
2.2.3 Pengaruh Learning Rate.....	18
2.2.4 Pengaruh Inisialisasi Bobot.....	22
2.2.5 Pengaruh Regularisasi.....	24
2.2.6 Perbandingan Dengan Library Scikit-Learn.....	28
III. Kesimpulan dan Saran.....	29
3.1 Kesimpulan.....	29
3.2 Saran.....	29
REFERENSI.....	30
LAMPIRAN.....	31
Kontribusi Tiap Anggota.....	31
Tautan GitHub.....	31
SURAT PERNYATAAN PENGGUNAAN AI.....	32

I. Deskripsi Persoalan

Tugas Besar I pada kuliah IF3270 Pembelajaran Mesin memiliki tujuan agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan Feedforward Neural Network (FFNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan FFNN from scratch. Peserta kuliah diminta untuk mengimplementasikan suatu modul FFNN yang memenuhi ketentuan-ketentuan berikut:

- FFNN yang diimplementasikan dapat menerima jumlah neuron dari tiap layer (termasuk input layer dan output layer).
- FFNN yang diimplementasikan dapat menerima fungsi aktivasi dari tiap layer. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

- FFNN yang diimplementasikan dapat **menerima fungsi loss** dari model tersebut. Pilihan loss function yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
----------------------	-----------------

MSE	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
Binary Cross-Entropy	$\mathcal{L}_{BCE} = -\frac{1}{n} \sum_{i=1}^n (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
Categorical Cross-Entropy	$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C (y_{ij} \log \hat{y}_{ij})$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size

- Catatan:
 - Binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2.
 - Log yang digunakan merupakan logaritma natural (logaritma dengan basis e).
- Terdapat mekanisme untuk **inisialisasi bobot** tiap neuron (termasuk bias). Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah sebagai berikut:
 - **Zero initialization**
 - Random dengan distribusi **uniform**.
 - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal).
 - Menerima parameter seed untuk reproducibility.
 - Random dengan distribusi **normal**.
 - Menerima parameter mean dan variance.
 - Menerima parameter seed untuk reproducibility.

- Instance model yang diinisialisasikan harus bisa **menyimpan bobot** tiap neuron (termasuk bias).
- Instance model yang diinisialisasikan harus bisa **menyimpan gradien bobot** tiap neuron (termasuk bias).
- Instance model memiliki method untuk **menampilkan distribusi bobot** dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk **menampilkan distribusi gradien bobot** dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk *save* dan *load*.
- Model memiliki implementasi **forward propagation** dengan ketentuan sebagai berikut:
 - Dapat menerima input berupa **batch**.
- Model memiliki implementasi **backward propagation** untuk menghitung perubahan gradien:
 - Dapat menangani perhitungan perubahan gradien untuk input data **batch**.
 - Gunakan konsep **chain rule** untuk menghitung gradien tiap bobot terhadap loss function.
 - Berikut merupakan **turunan pertama** untuk setiap fungsi aktivasi:

Nama Fungsi Aktivasi	Turunan Pertama
Linear	$\frac{d(\text{Linear}(x))}{dx} = 1$
ReLU	$\frac{d(\text{ReLU}(x))}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d(\sigma(x))}{dx} = \sigma(x)(1 - \sigma(x))$
Hyperbolic Tangent (tanh)	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x - e^{-x}} \right)^2$

Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $\frac{d(\text{softmax}(\vec{x}))}{d\vec{x}} = \begin{bmatrix} \frac{\partial(\text{softmax}(\vec{x}))_1}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x}))_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(\text{softmax}(\vec{x}))_n}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x}))_n}{\partial x_n} \end{bmatrix}$ Dimana untuk $i, j \in \{1, \dots, n\}$, $\frac{\partial(\text{softmax}(\vec{x}))_i}{\partial x_j} = \text{softmax}(\vec{x})_i (\delta_{ij} - \text{softmax}(\vec{x})_j)$ $\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$
---------	--

- Berikut merupakan **turunan pertama** untuk setiap fungsi loss terhadap bobot suatu FFNN (lanjutkan sisanya menggunakan chain rule):

Nama Fungsi Loss	Definisi Fungsi
MSE	$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \frac{\partial \hat{y}_i}{\partial W}$
Binary Cross-Entropy	$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \frac{\partial \hat{y}_i}{\partial W}$
Categorical Cross-Entropy	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C \frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C \frac{y_{ij}}{\hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W}$

- Model memiliki implementasi metode **regularisasi L1** dan **L2**.
- Model memiliki implementasi **weight update** dengan menggunakan **gradient descent** untuk memperbarui bobot berdasarkan gradien yang telah dihitung, berikut persamaannya:

$$W_{new} = W_{old} - \alpha \left(\frac{\partial \mathcal{L}}{\partial W_{old}} \right)$$

α = Learning rate

- Implementasi untuk pelatihan model harus memenuhi ketentuan berikut:
 - Dapat menerima parameter berikut:

- Batch Size
- Learning Rate
- Jumlah Epoch
- Verbose
- Verbose 0 berarti tidak menampilkan apa-apa selama pelatihan.
- Verbose 1 berarti hanya menampilkan progress bar beserta dengan kondisi training loss dan validation loss saat itu.
 - Proses pelatihan mengembalikan **histori dari proses pelatihan** yang berisi **training loss** dan **validation loss** tiap epoch.

II. Pembahasan

2.1 Penjelasan Implementasi

2.1.1 Implementasi Kelas

Program dibangun dengan menggunakan 2 file kelas utama, yaitu `functions.py`, `ffnn.py`, serta `main.ipynb`. File `functions.py` dan `ffnn.py` berisi kelas-kelas dan fungsi-fungsi yang penting untuk FFNN, sedangkan `main.ipynb` hanya merupakan main program untuk menjalankan model dan pengujian.

File `functions.py` berfungsi untuk menyimpan kelas-kelas fungsi aktivasi (Linear, ReLU, Sigmoid, Tanh, dan Softmax) yang masing-masing berisi fungsi `f` dan `df_dx` dengan atribut `x` : `np.ndarray` dan return berupa `np.ndarray` juga. File tersebut juga berisi kelas-kelas error (MSE, BCE, dan CCE) yang berisi `L` dan `dL_do` dengan atribut `t` : `np.ndarray` dan `o` : `np.ndarray` dan return berupa `np.ndarray` juga.

File `ffnn.py` berfungsi untuk menyimpan kelas-kelas yang berkaitan dengan FFNN, mulai dari forward propagation, hingga backward propagation. Terdapat 9 fungsi utama pada kelas ini, yaitu `__init__` untuk inisialisasi awal, `count_net_per_layer` untuk menghitung net value setiap neuron di layer tertentu, `activate_layer` untuk mengaktivasi suatu layer dengan fungsi aktivasinya, `predict` untuk melakukan forward propagation, `result` untuk mengambil output akhirnya, `compute_delta_output_layer` untuk menghitung delta di output layer, `compute_delta_hidden_layer` untuk menghitung delta di hidden layer, `update_weight` untuk melakukan update bobot pada setiap epoch, `train` untuk melakukan proses pembelajaran, serta beberapa fungsi plot tambahan untuk analisis grafik dan loss.

2.1.2 Implementasi Forward Propagation

Forward propagation merupakan tahap awal dari FFNN dengan melakukan propagasi secara maju dari input layer menuju output layer dengan melalui sejumlah hidden layer. Pada implementasi yang dilakukan, ini dimulai dengan inisialisasi FFNN yang memiliki berapa unit (neuron) per layer (mulai dari input layer hingga output layer), serta berbagai parameter pendukung lainnya.

Pada tahap berikutnya, setelah melakukan inisialisasi FFNN, forward propagation dimulai dengan hidden layer pertama menghitung net valuenya berdasarkan bobot dan value dari setiap unit pada input layer dengan menggunakan fungsi `count_net_per_layer`. Selanjutnya, hasil net value pada hidden layer pertama tersebut diaktivasi agar menjadi value tertentu dengan

menggunakan fungsi aktivasi yang telah ditentukan menggunakan fungsi `activate_layer`.

Proses tersebut dilakukan terus menerus secara maju (input layer dengan hidden layer ke-1, hidden layer ke-1 dengan hidden layer ke-2, dan seterusnya hingga hidden layer terakhir dengan output layer). Proses forward propagation ini dilakukan menggunakan fungsi `predict` dan hasil akhirnya pada output layer diambil dengan fungsi `result`.

2.1.3 Implementasi Backward Propagation

Backward propagation merupakan tahap berikutnya dari FFNN dengan melakukan propagasi secara mundur dari output layer menuju hidden layer pertama dengan melalui sejumlah hidden layer lainnya. Pada implementasi yang dilakukan, hal ini dimulai dengan hasil dari forward propagation sebelumnya.

Pada tahap berikutnya, dilakukan perhitungan delta value untuk setiap unit pada output layer dengan fungsi `compute_delta_output_layer`. Di balik layar, proses ini dilakukan dengan menghitung dL_{do} , lalu mengalikan turunan dari fungsi aktivasi dengan dL_{do} tersebut. Lalu, dihitung delta value pada setiap hidden layer secara mundur (dari hidden layer terakhir ke hidden layer pertama) dengan fungsi `compute_delta_hidden_layer` dengan cara yang sama.

Setelah menghitung delta dari setiap layer, tahap terakhir dari backward propagation adalah melakukan update bobot pada fungsi `update_weight`. Proses pembaruan bobot ini dilakukan dengan mengurangi weight lamanya dengan learning rate dikali dengan gradien bobot yang telah ditambah dengan regularisasi L1 dan/atau regularisasi L2. Pada akhirnya, 1 epoch selesai dan dilanjutkan ulang dengan forward propagation kembali untuk epoch berikutnya.

2.2 Hasil Pengujian

2.2.1 Pengaruh Depth dan Width

Pada pengujian ini, akan diberikan 3 pasang depth (banyak hidden layer), yaitu 1, 2, dan 4, serta 3 pasang width (banyak neuron dalam setiap hidden layer), yaitu 8, 16, dan 32. Berikut adalah hasil pengujiannya :

- Depth 1 & Width 8 :

```

=====
[WIDTH] w=8
=====
              precision    recall  f1-score   support

not placed (0)      0.67      0.57      0.62       775
  placed (1)       0.75      0.83      0.79      1225

    accuracy              0.73      2000
   macro avg       0.71      0.70      0.70      2000
  weighted avg       0.72      0.73      0.72      2000

>>> Macro F1-Score: 0.7016

```

- Depth 1 & Width 16 :

```

=====
[WIDTH] w=16
=====
              precision    recall  f1-score   support

not placed (0)      0.69      0.40      0.51       775
  placed (1)       0.70      0.89      0.78      1225

    accuracy              0.70      2000
   macro avg       0.70      0.64      0.65      2000
  weighted avg       0.70      0.70      0.68      2000

>>> Macro F1-Score: 0.6452

```

- Depth 1 & Width 32 :

```

=====
[WIDTH] w=32
=====
              precision    recall  f1-score   support

not placed (0)      0.62      0.59      0.60       775
  placed (1)       0.75      0.77      0.76      1225

    accuracy              0.70      2000
   macro avg       0.68      0.68      0.68      2000
  weighted avg       0.70      0.70      0.70      2000

>>> Macro F1-Score: 0.6817

```

- Depth 2 & Width 8 :

```

=====
[WIDTH] w=8
=====
              precision    recall  f1-score   support

not placed (0)      0.28      0.14      0.19       775
  placed (1)       0.59      0.78      0.67      1225

    accuracy              0.53      2000
   macro avg       0.44      0.46      0.43      2000
  weighted avg     0.47      0.53      0.48      2000

>>> Macro F1-Score: 0.4276

```

- Depth 2 & Width 16 :

```

=====
[WIDTH] w=16
=====
              precision    recall  f1-score   support

not placed (0)      0.46      0.39      0.42       775
  placed (1)       0.65      0.71      0.68      1225

    accuracy              0.59      2000
   macro avg       0.55      0.55      0.55      2000
  weighted avg     0.58      0.59      0.58      2000

>>> Macro F1-Score: 0.5498

```

- Depth 2 & Width 32 :

```

=====
[WIDTH] w=32
=====
              precision    recall  f1-score   support

not placed (0)      0.69      0.48      0.56       775
  placed (1)       0.72      0.86      0.79      1225

    accuracy              0.71      2000
   macro avg       0.71      0.67      0.68      2000
  weighted avg     0.71      0.71      0.70      2000

>>> Macro F1-Score: 0.6761

```

- Depth 4 & Width 8 :

```

=====
[WIDTH] w=8
=====
              precision    recall  f1-score   support

not placed (0)      0.33      0.45      0.38       775
  placed (1)       0.54      0.41      0.47      1225

   accuracy              0.43      2000
  macro avg       0.44      0.43      0.42      2000
 weighted avg       0.46      0.43      0.44      2000

>>> Macro F1-Score: 0.4250

```

- Depth 4 & Width 16 :

```

=====
[WIDTH] w=16
=====
              precision    recall  f1-score   support

not placed (0)      0.50      0.62      0.56       775
  placed (1)       0.72      0.61      0.66      1225

   accuracy              0.62      2000
  macro avg       0.61      0.62      0.61      2000
 weighted avg       0.64      0.62      0.62      2000

>>> Macro F1-Score: 0.6087

```

- Depth 4 & Width 32 :

```

=====
[WIDTH] w=32
=====
              precision    recall  f1-score   support

not placed (0)      0.39      1.00      0.56       775
  placed (1)       0.00      0.00      0.00      1225

   accuracy              0.39      2000
  macro avg       0.19      0.50      0.28      2000
 weighted avg       0.15      0.39      0.22      2000

>>> Macro F1-Score: 0.2793

```

Berdasarkan hasil pengujian tersebut, ditemukan bahwa performa sangat bervariasi terhadap perbedaan width dan depth. Performa terbaik didapat dari width 8 dan depth 1, sedangkan yang terburuk adalah depth 4 dan width 32.

2.2.2 Pengaruh Fungsi Aktivasi

Pada pengujian ini, akan diberikan 1 layer untuk diuji fungsi aktivasinya, sedangkan layer lainnya tetap menggunakan fungsi aktivasi yang sama, yaitu layer ke-1. Berikut adalah hasil pengujiannya :

- Fungsi Aktivasi Linear :

```
=====
[ACT] h1=Linear
=====
              precision    recall  f1-score   support

not placed (0)        0.64      0.77      0.70        775
placed (1)           0.83      0.73      0.78       1225

   accuracy
macro avg          0.74      0.75      0.74       2000
weighted avg       0.76      0.75      0.75       2000

>>> Macro F1-Score: 0.7403
```

- Fungsi Aktivasi ReLU :

```
=====
[ACT] h1=ReLU
=====
              precision    recall  f1-score   support

not placed (0)        0.69      0.67      0.68        775
placed (1)           0.79      0.81      0.80       1225

   accuracy
macro avg          0.74      0.74      0.74       2000
weighted avg       0.76      0.76      0.76       2000

>>> Macro F1-Score: 0.7423
```

- Fungsi Aktivasi Sigmoid :

```
=====
[ACT] h1=Sigmoid
=====
              precision    recall  f1-score   support

not placed (0)        0.77      0.54      0.63        775
placed (1)           0.75      0.90      0.82       1225

   accuracy
macro avg          0.76      0.72      0.73       2000
weighted avg       0.76      0.76      0.75       2000

>>> Macro F1-Score: 0.7259
```

- Fungsi Aktivasi Hyperbolic Tangent (tanh) :

```

=====
[ACT] h1=Sigmoid
=====

```

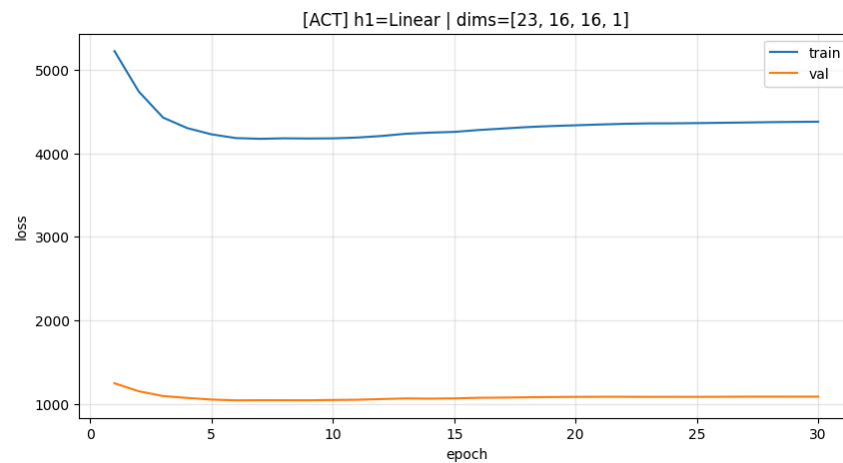
	precision	recall	f1-score	support
not placed (0)	0.77	0.54	0.63	775
placed (1)	0.75	0.90	0.82	1225
accuracy			0.76	2000
macro avg	0.76	0.72	0.73	2000
weighted avg	0.76	0.76	0.75	2000

```

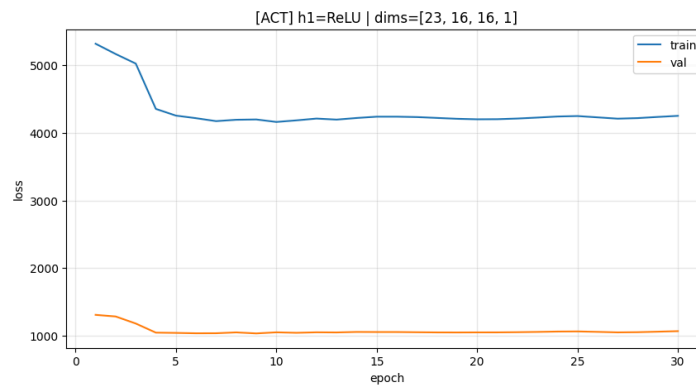
>>> Macro F1-Score: 0.7259

```

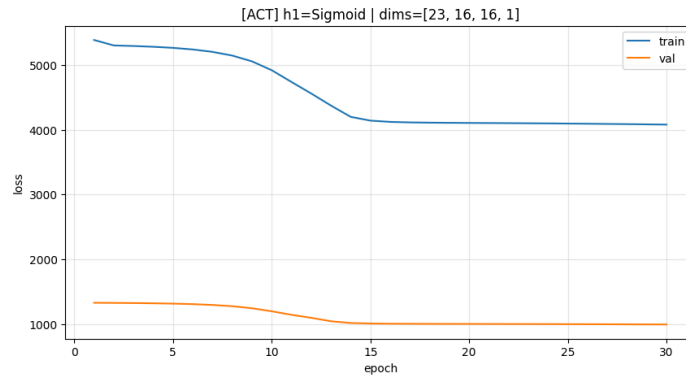
- Grafik Training Loss dan Validation Loss Fungsi Aktivasi Linear :



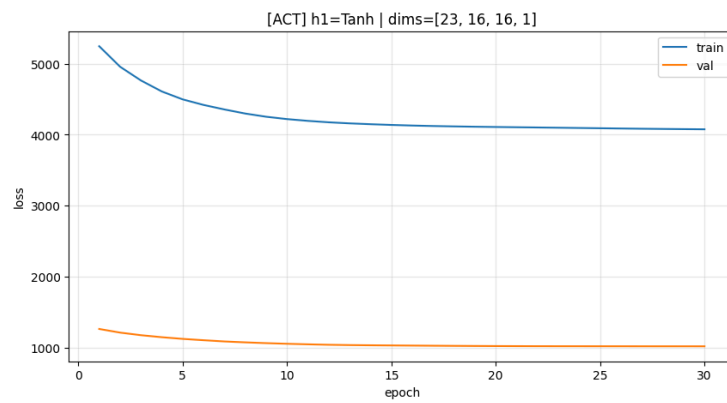
- Grafik Training Loss dan Validation Loss Fungsi Aktivasi ReLU :



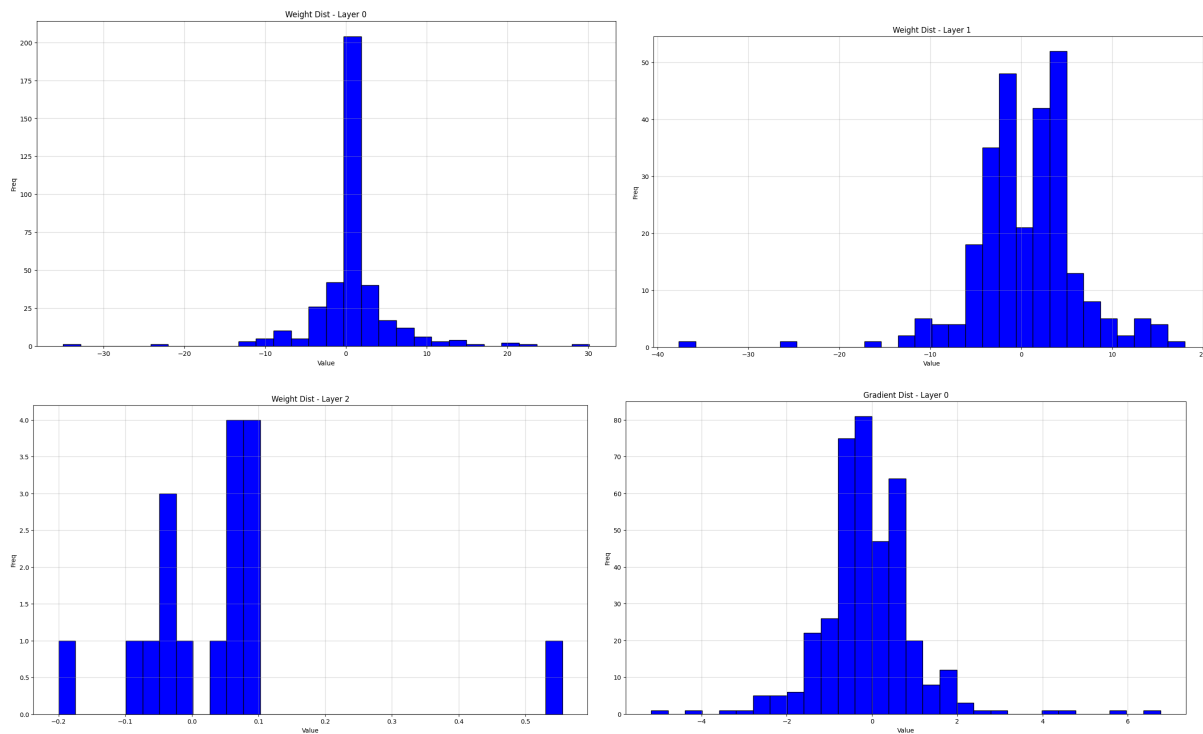
- Grafik Training Loss dan Validation Loss Fungsi Aktivasi Sigmoid :

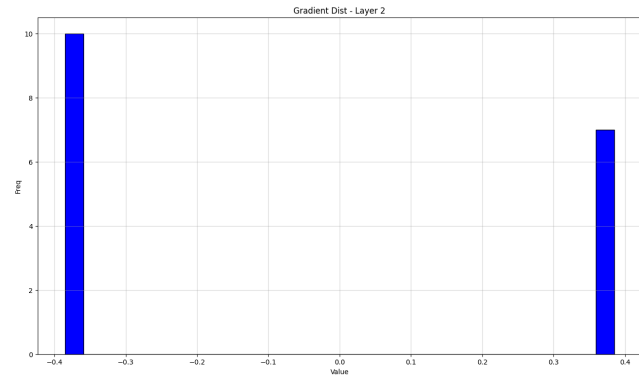
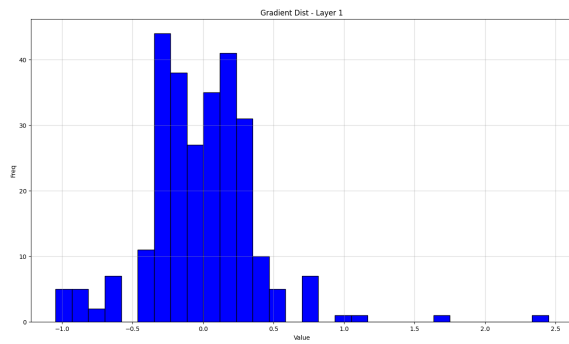


- Grafik Training Loss dan Validation Loss Fungsi Aktivasi Hyperbolic Tangent (tanh) :

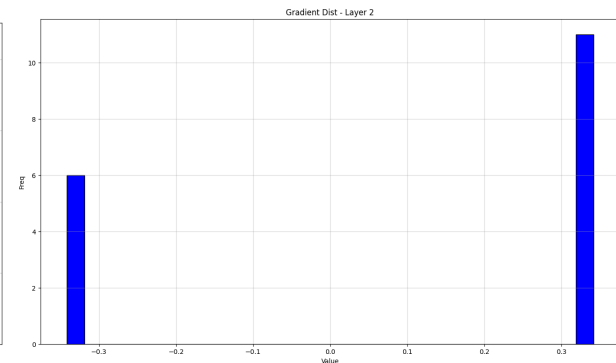
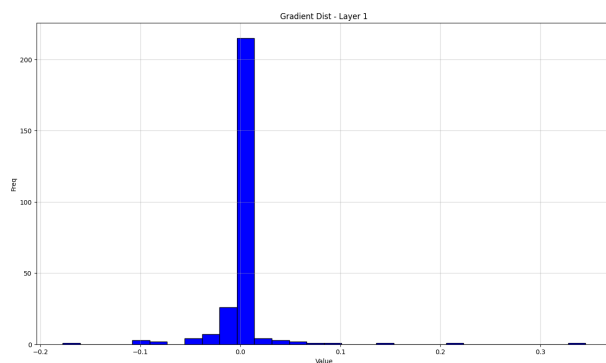
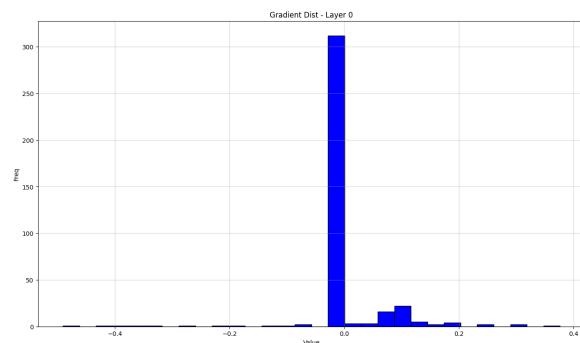
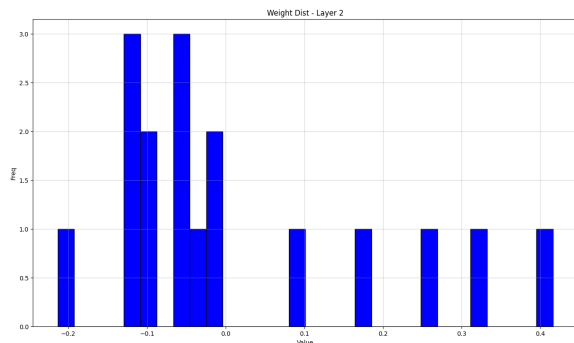
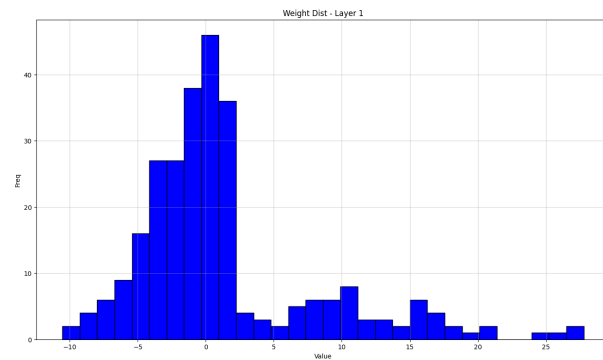
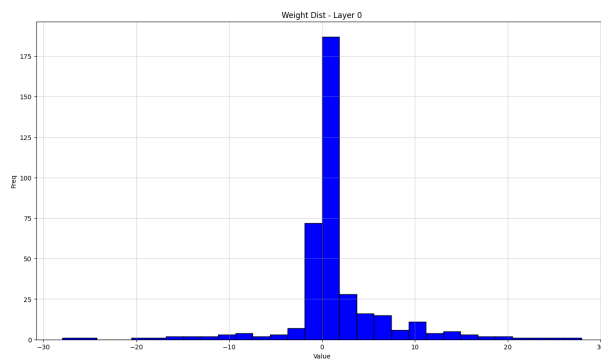


- Distribusi Bobot dan Gradien Bobot Fungsi Aktivasi Linear :

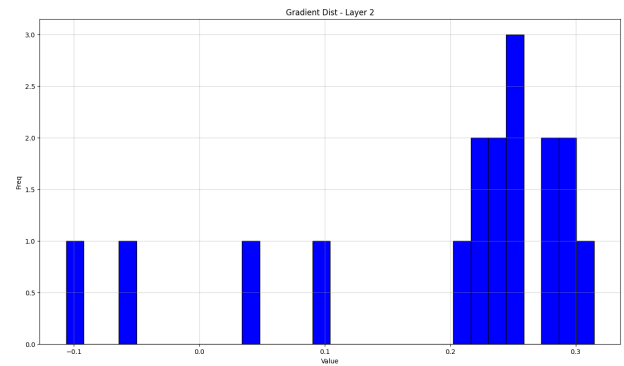
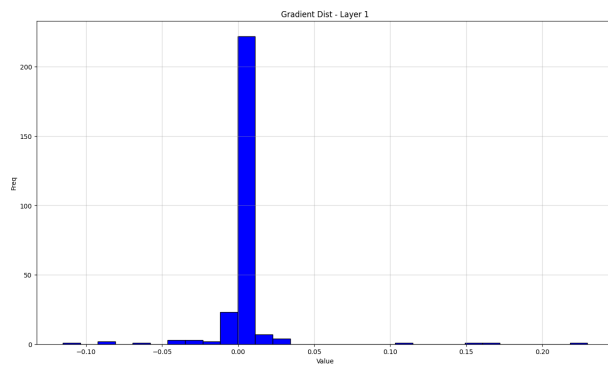
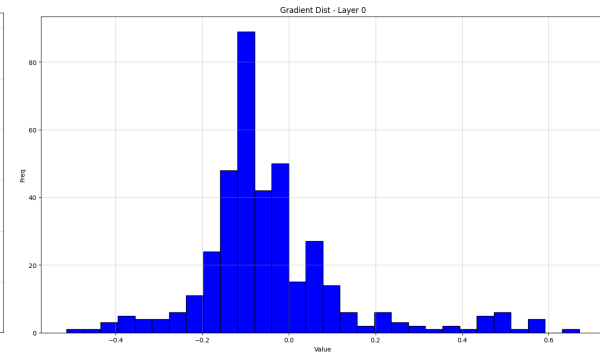
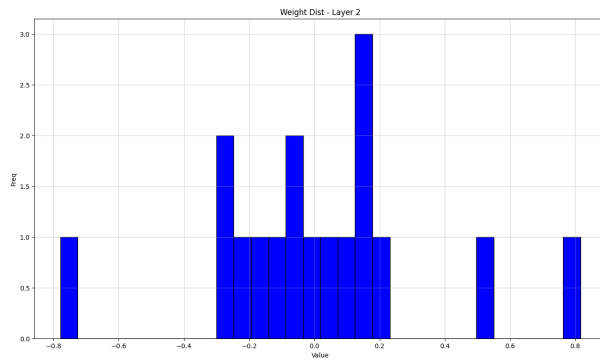
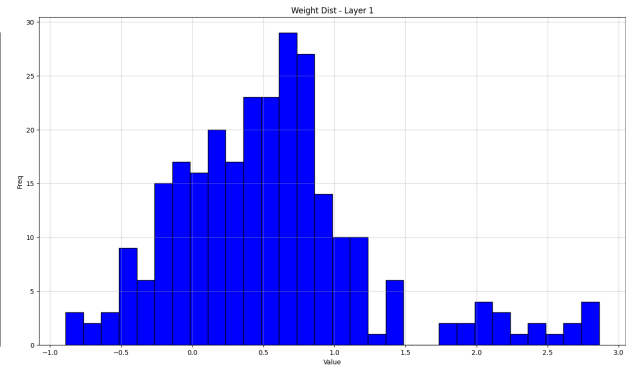
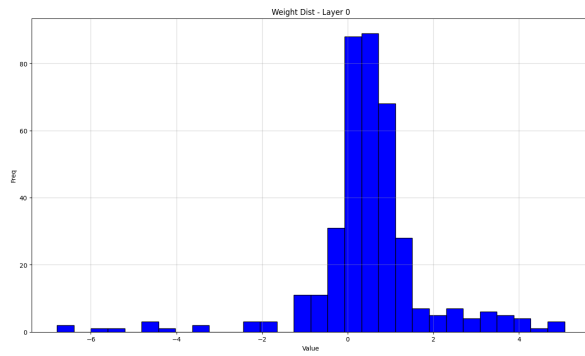




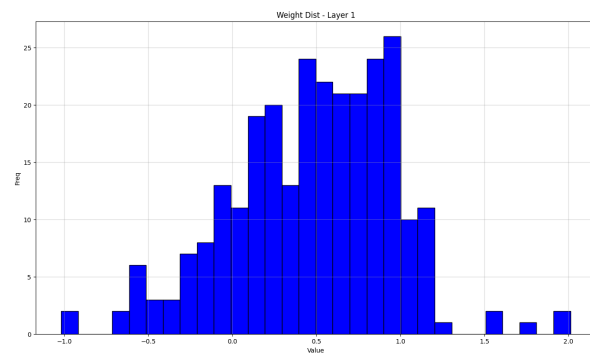
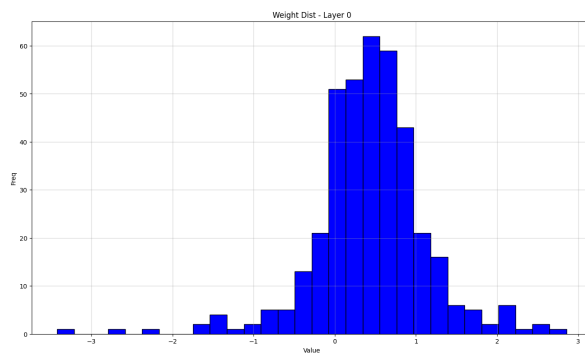
- Distribusi Bobot dan Gradien Bobot Fungsi Aktivasi ReLU :

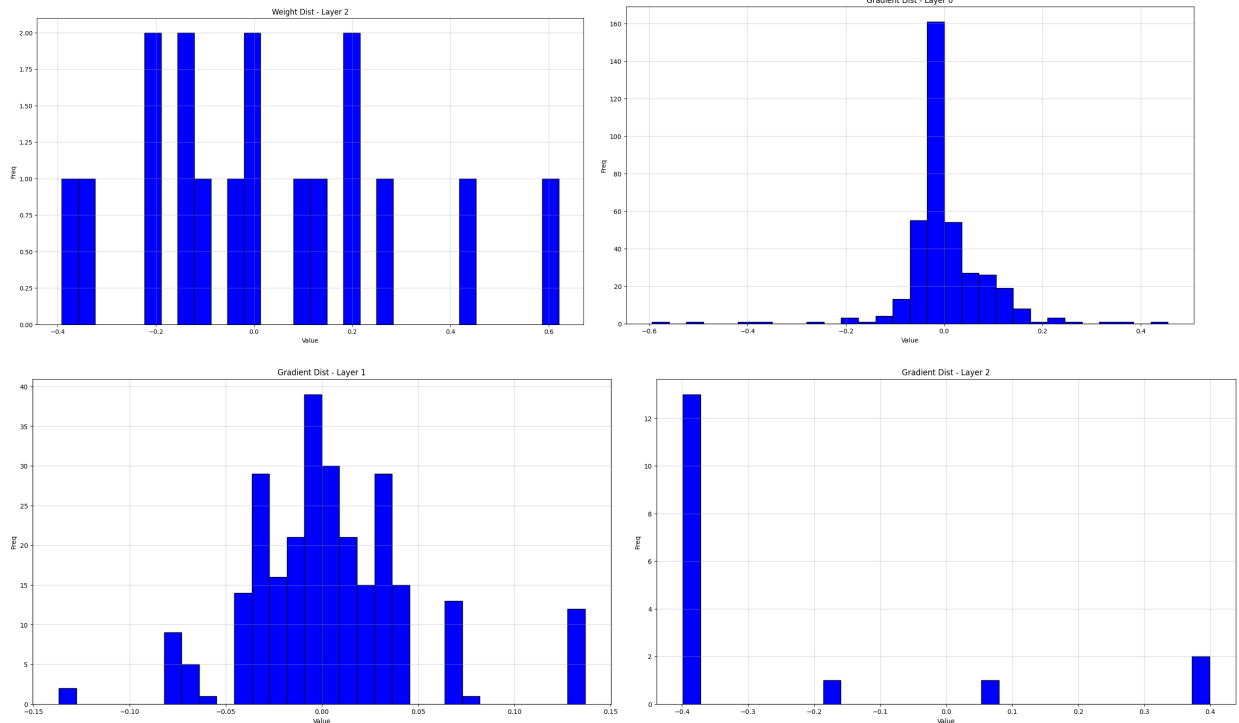


- Distribusi Bobot dan Gradien Bobot Fungsi Aktivasi Sigmoid :



- Distribusi Bobot dan Gradien Bobot Fungsi Aktivasi Hyperbolic Tangent (tanh) :





Berdasarkan hasil pengujian tersebut, ditemukan bahwa fungsi aktivasi ReLU memiliki performa yang terbaik, dan fungsi sigmoid dan tanh memiliki performa yang paling buruk. Namun, keempat fungsi aktivasi memiliki performa yang sangat mirip.

2.2.3 Pengaruh Learning Rate

Pada pengujian ini, akan diberikan 3 pasang nilai learning rate, yaitu 0.001, 0.01, dan 0.1. Berikut adalah hasil pengujiannya :

- Learning Rate 0.001 :

```
=====
[LR] lr=0.001
=====
              precision    recall  f1-score   support

not placed (0)       0.70      0.63      0.66         775
placed (1)           0.78      0.83      0.80        1225

   accuracy                   0.75         2000
  macro avg              0.74      0.73      0.73         2000
 weighted avg              0.75      0.75      0.75         2000

>>> Macro F1-Score: 0.7345
```

- Learning Rate 0.01 :

```

=====
[LR] lr=0.01
=====
              precision    recall  f1-score   support

not placed (0)    0.66      0.63      0.65       775
placed (1)        0.77      0.80      0.78      1225

   accuracy              0.73      2000
  macro avg    0.72      0.71      0.72      2000
 weighted avg    0.73      0.73      0.73      2000

>>> Macro F1-Score: 0.7158

```

- Learning Rate 0.1 :

```

=====
[LR] lr=0.1
=====
              precision    recall  f1-score   support

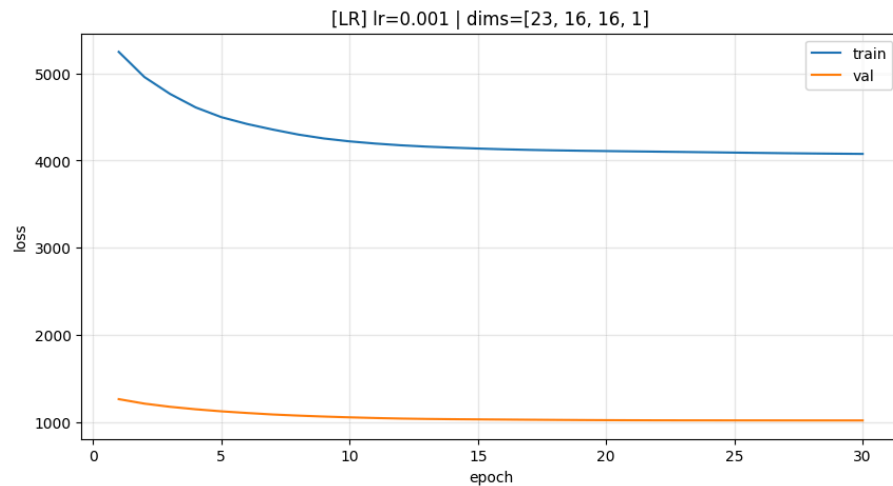
not placed (0)    0.46      0.39      0.42       775
placed (1)        0.65      0.71      0.68      1225

   accuracy              0.59      2000
  macro avg    0.55      0.55      0.55      2000
 weighted avg    0.58      0.59      0.58      2000

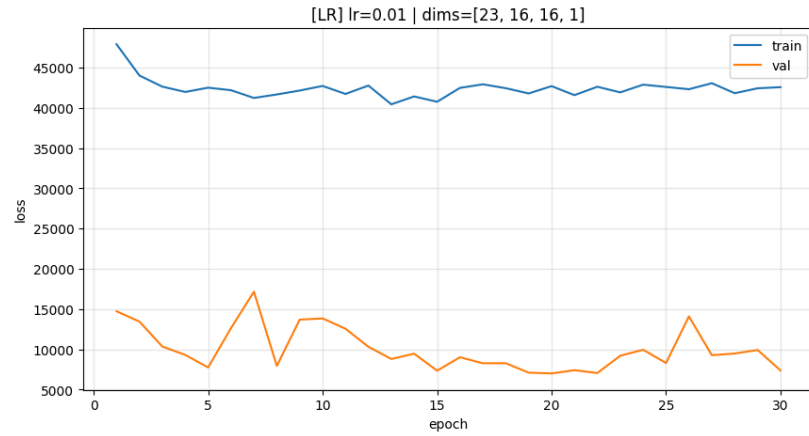
>>> Macro F1-Score: 0.5498

```

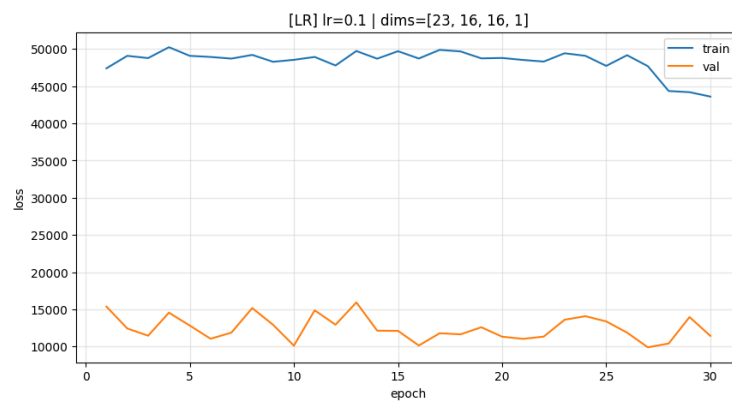
- Grafik Training Loss dan Validation Loss Learning Rate 0.001 :



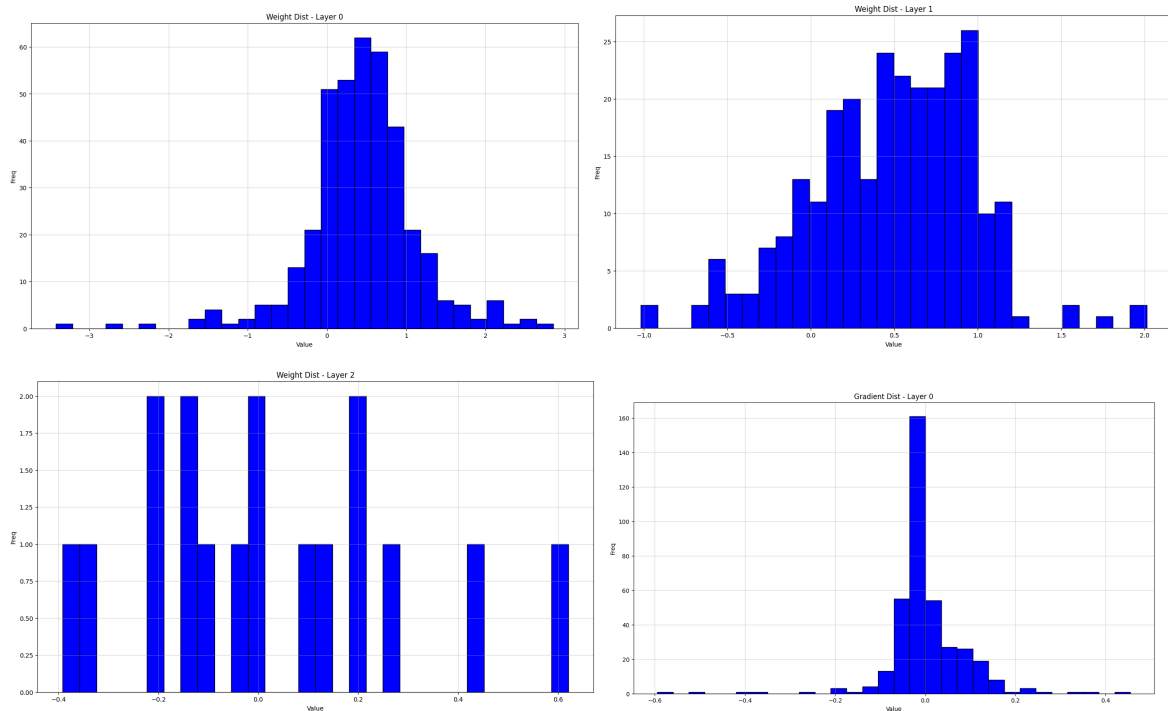
- Grafik Training Loss dan Validation Loss Learning Rate 0.01 :

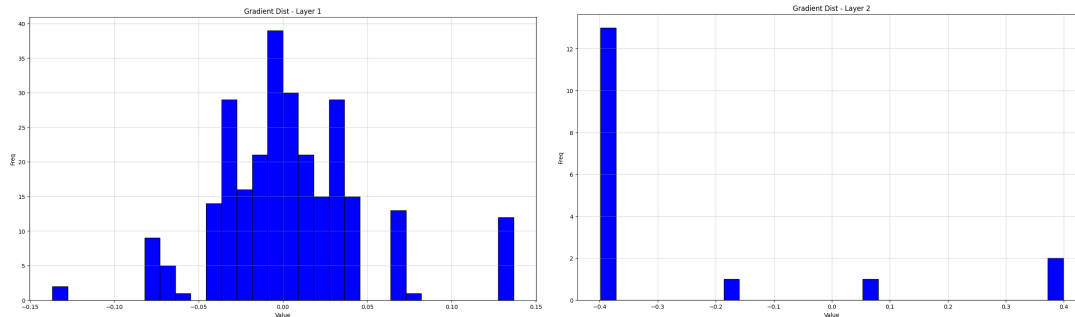


- Grafik Training Loss dan Validation Loss Learning Rate 0.1 :

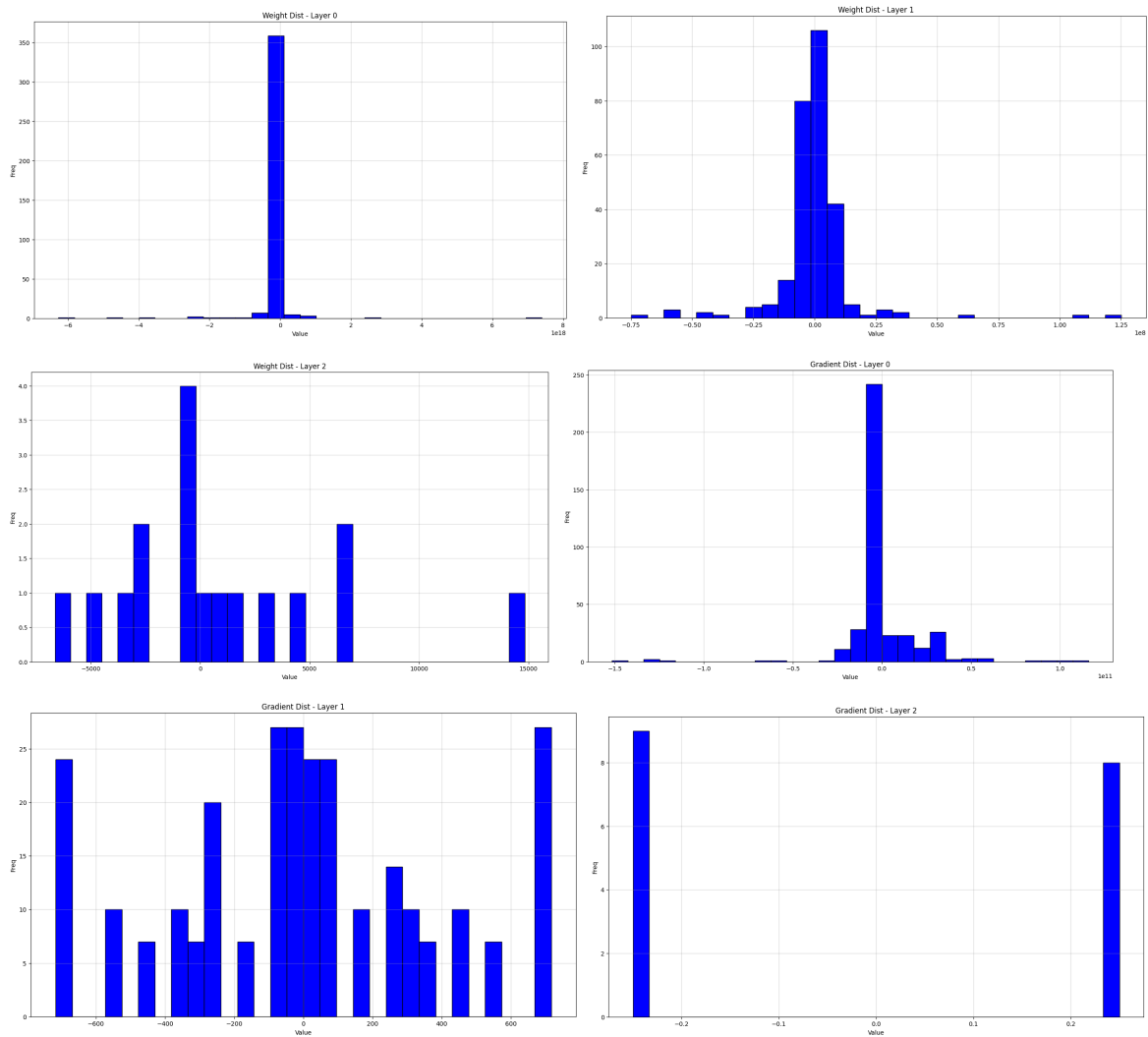


- Distribusi Bobot dan Gradien Bobot Learning Rate 0.001 :

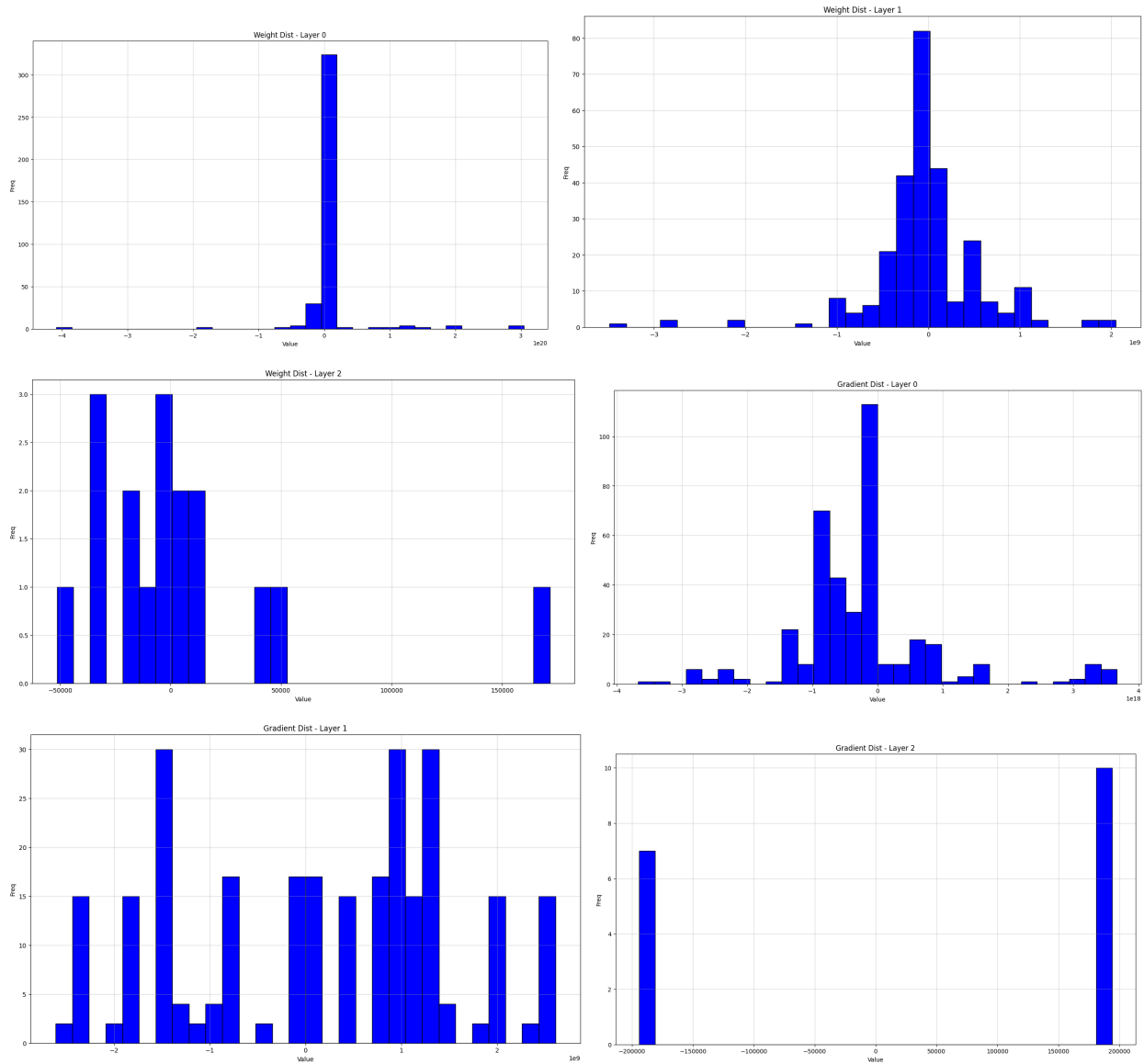




- Distribusi Bobot dan Gradien Bobot Learning Rate 0.01 :



- Distribusi Bobot dan Gradien Bobot Learning Rate C :



Berdasarkan hasil pengujian tersebut, ditemukan bahwa semakin kecil learning rate maka semakin bagus pula performa modelnya. Hal ini konsisten dengan hasil percobaan. distribusi weight juga tersebar lebih luas pada learning rate yang kecil, sedangkan pada learning rate yang besar weight terpusat di nol (sambungan neuron cenderung banyak yang “mati”).

2.2.4 Pengaruh Inisialisasi Bobot

Pada pengujian ini, akan diberikan 5 buah inisialisasi bobot, yaitu zero, uniform, normal distribution, xavier, dan he initialization. Berikut adalah hasil pengujiannya :

- Inisialisasi Bobot zero:

```

=====
[LR] lr=0.001
=====
              precision    recall  f1-score   support

not placed (0)      0.00      0.00      0.00        775
  placed (1)       0.61      1.00      0.76       1225

   accuracy              0.61       2000
  macro avg       0.31      0.50      0.38       2000
 weighted avg       0.38      0.61      0.47       2000

>>> Macro F1-Score: 0.3798

```

- Inisialisasi Bobot uniform uniform :

```

=====
[LR] lr=0.001
=====
              precision    recall  f1-score   support

not placed (0)      0.70      0.63      0.66        775
  placed (1)       0.78      0.83      0.80       1225

   accuracy              0.75       2000
  macro avg       0.74      0.73      0.73       2000
 weighted avg       0.75      0.75      0.75       2000

>>> Macro F1-Score: 0.7345

```

- Inisialisasi Bobot normal distribution :

```

=====
[LR] lr=0.001
=====
              precision    recall  f1-score   support

not placed (0)      0.69      0.66      0.67        775
  placed (1)       0.79      0.81      0.80       1225

   accuracy              0.75       2000
  macro avg       0.74      0.74      0.74       2000
 weighted avg       0.75      0.75      0.75       2000

>>> Macro F1-Score: 0.7376

```

- Inisialisasi Bobot Xavier :

```

=====
[LR] lr=0.001
=====
              precision    recall  f1-score   support

not placed (0)      0.73      0.59      0.65       775
placed (1)          0.77      0.86      0.81      1225

   accuracy          0.76      2000
  macro avg          0.75      0.73      0.73      2000
 weighted avg          0.75      0.76      0.75      2000

>>> Macro F1-Score: 0.7335

```

- Inisialisasi Bobot He:

```

=====
[LR] lr=0.001
=====
              precision    recall  f1-score   support

not placed (0)      0.71      0.61      0.66       775
placed (1)          0.77      0.84      0.81      1225

   accuracy          0.75      2000
  macro avg          0.74      0.73      0.73      2000
 weighted avg          0.75      0.75      0.75      2000

>>> Macro F1-Score: 0.7301

```

Berdasarkan hasil pengujian tersebut, ditemukan bahwa metode inisialisasi yang paling baik adalah distribusi normal dengan mean 0 dan deviasi standar 1.

2.2.5 Pengaruh Regularisasi

Pada pengujian ini, akan diberikan 3 aturan regularisasi, yaitu tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2. Berikut adalah hasil pengujiannya :

- Tanpa Regularisasi :

```

Training: 100%|██████████| 30/30 [00:18<00:00, 1.60it/s, loss=42562.976562]
=====
[REG] none
=====
              precision    recall  f1-score   support

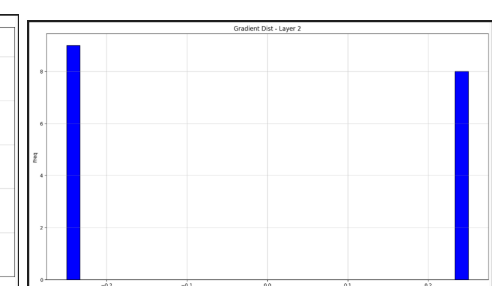
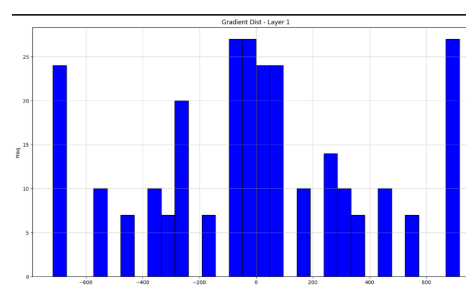
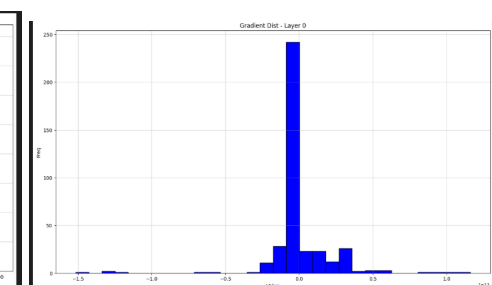
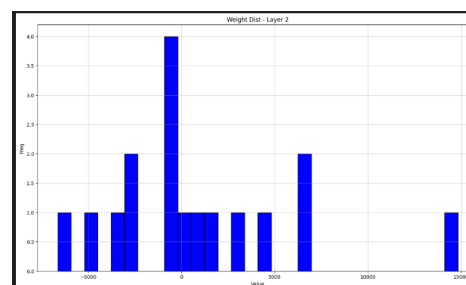
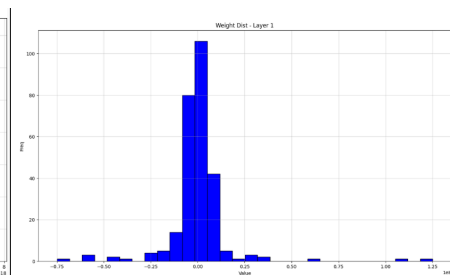
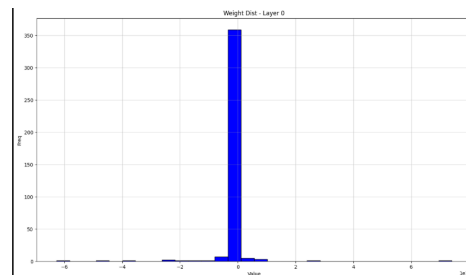
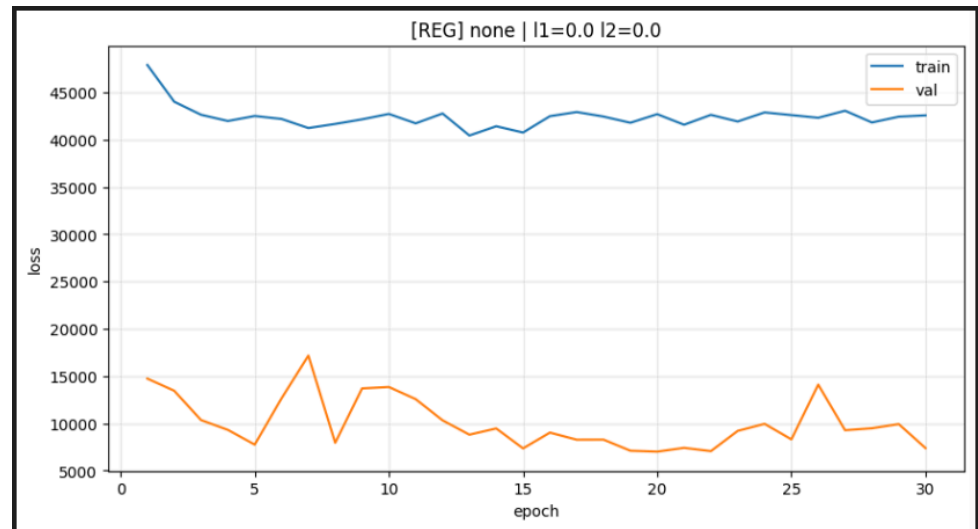
not placed (0)      0.66      0.63      0.65       775
placed (1)          0.77      0.80      0.78      1225

   accuracy          0.73      2000
  macro avg          0.72      0.71      0.72      2000
 weighted avg          0.73      0.73      0.73      2000

>>> Macro F1-Score: 0.7158

```


- Grafik Training Loss dan Validation Loss Tanpa Regulasi



- Regularisasi L1 :

```

Training: 100%|██████████| 30/30 [00:17<00:00, 1.67it/s, loss=42831.058594]
=====
[REG] l1
=====

```

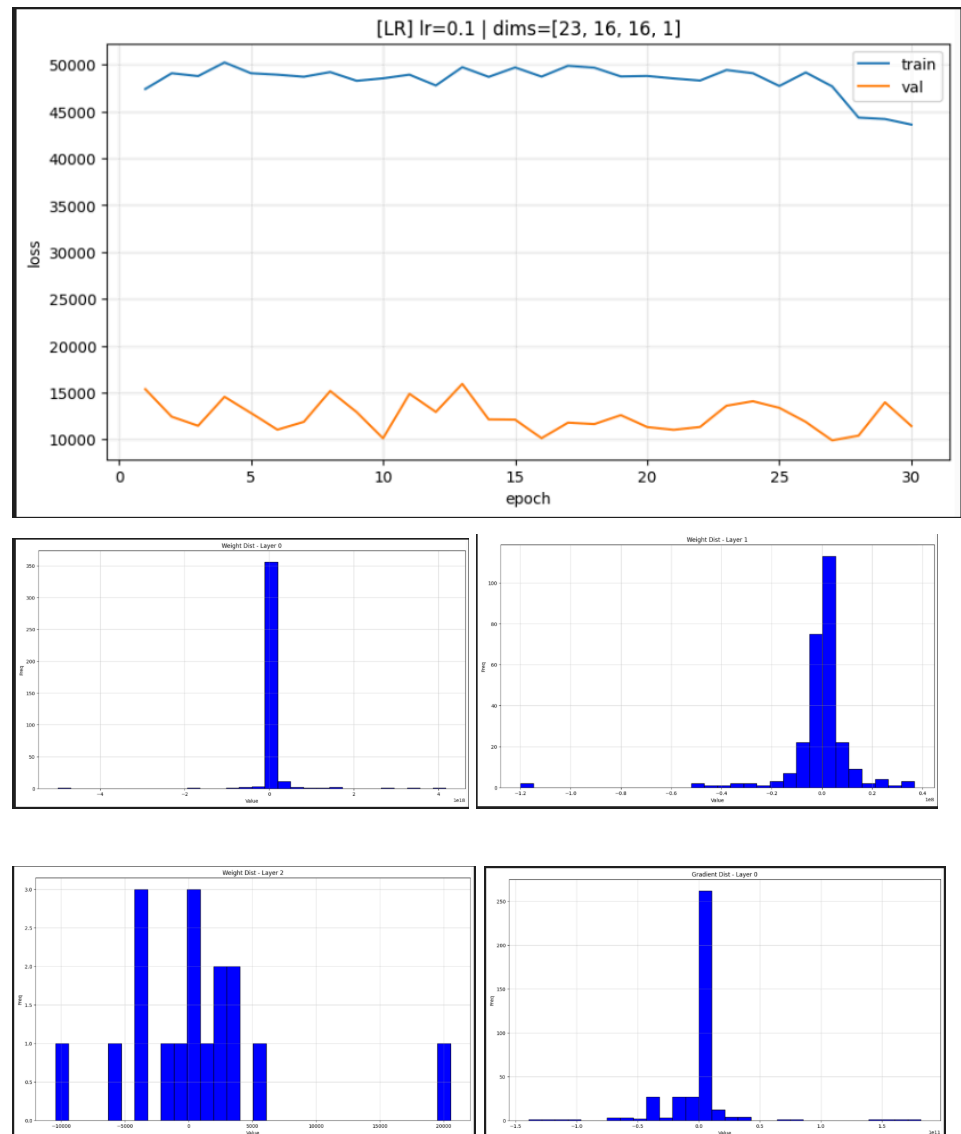
	precision	recall	f1-score	support
not placed (0)	0.62	0.30	0.40	775
placed (1)	0.67	0.88	0.76	1225
accuracy			0.66	2000
macro avg	0.64	0.59	0.58	2000
weighted avg	0.65	0.66	0.62	2000

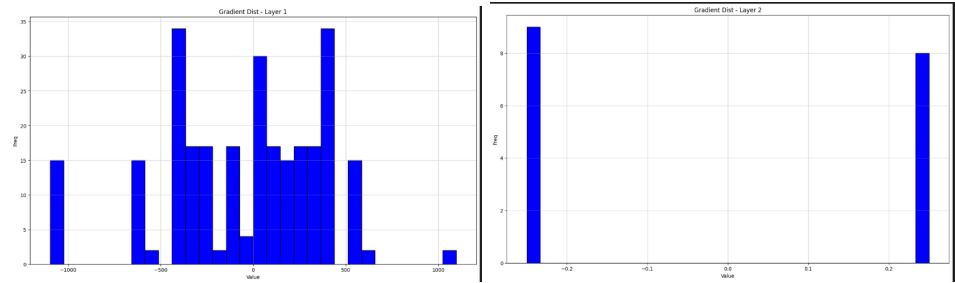
```

>>> Macro F1-Score: 0.5810

```

- Grafik Training Loss dan Validation Loss dengan regulasi l1



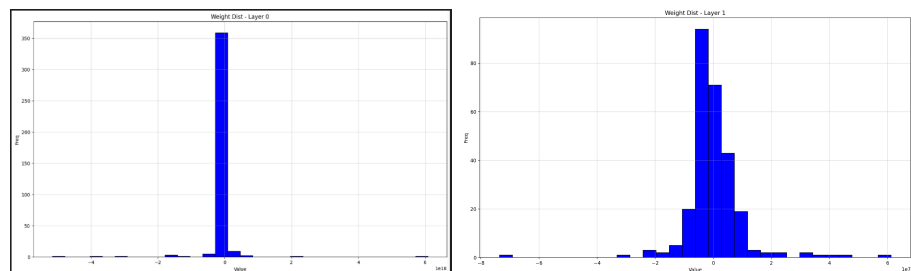
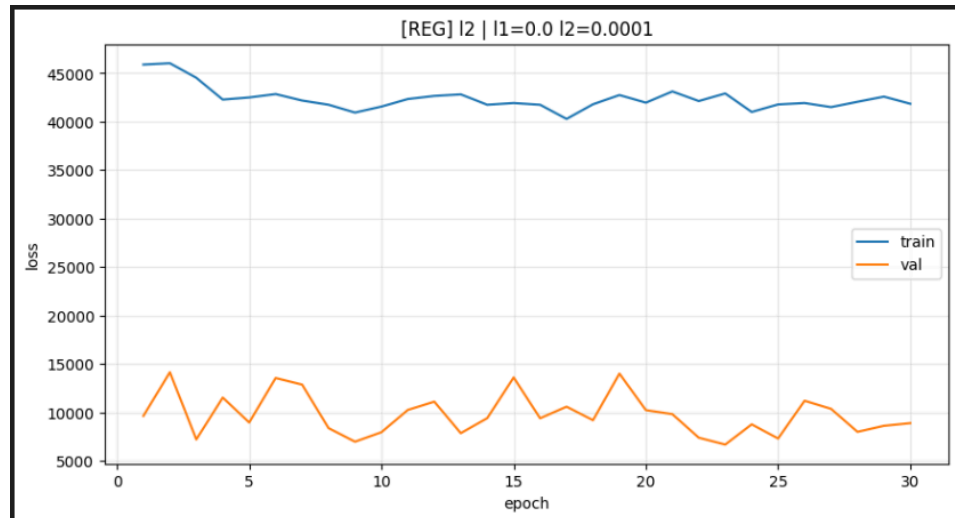


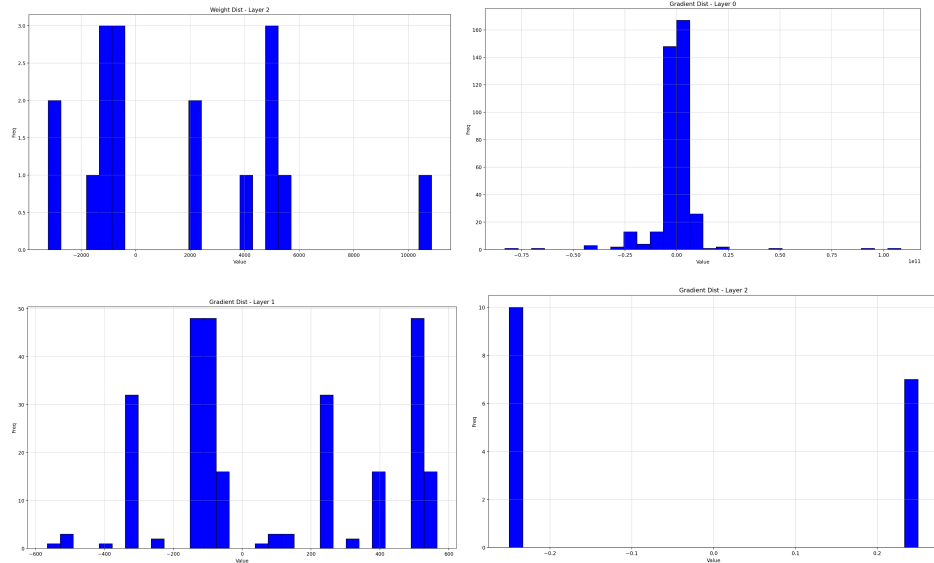
- Regularisasi L2 :

```
Training: 100% | 30/30 [00:16<00:00, 1.79it/s, loss=41850.640625]
[REG] l2
=====
precision    recall  f1-score   support
not placed (0)    0.65    0.37    0.47      775
placed (1)        0.69    0.87    0.77     1225

accuracy              0.68      2000
macro avg            0.67    0.62    0.62      2000
weighted avg         0.67    0.68    0.65      2000

>>> Macro F1-Score: 0.6199
```





Berdasarkan hasil pengujian tersebut, ditemukan bahwa **Ayam Goreng**.

2.2.6 Perbandingan Dengan Library Scikit-Learn

Pada pengujian ini, akan diberikan parameter yang sama untuk model FFNN from scratch dan juga model dari Scikit-Learn, yaitu A. Berikut adalah hasil pengujiannya :

```
Sklearn MLP macro_f1: 0.663808463923631
Training: 100%|██████████| 100/100 [00:48<00:00, 2.05it/s]
FFNN from scratch macro_f1: 0.6365517742687208
```

- FFNN from scratch :
0.6365517742687208
- Library Scikit-Learn :
0.663808463923631

Berdasarkan hasil pengujian tersebut, ditemukan bahwa dengan hyperparameter yang sama dapat menghasilkan yang berbeda mungkin dikarenakan.

III. Kesimpulan dan Saran

3.1 Kesimpulan

Tugas Besar (Tubes) 1 mata kuliah Pembelajaran Mesin (IF3270-24) ini berhasil mengimplementasikan Artificial Neural Network (ANN) - Feed Forward Neural Network (FFNN) from scratch. Pada tugas ini, peserta kuliah mampu mengimplementasikan seluruh bagian dari FFNN, mulai dari implementasi kelas, fungsi-fungsi, forward propagation, hingga backward propagation. Tidak hanya itu, peserta kuliah juga mampu melakukan visualisasi proses pembelajaran dengan baik serta melakukan preprocessing sebagaimana umumnya.

Implementasi kelas dan fungsi-fungsi yang dilakukan secara Object-Oriented Programming (OOP), membuat hasil dari FFNN yang telah dibuat menjadi lebih rapi dan terstruktur. Implementasi forward propagation dilakukan dengan menghitung nilai output dari setiap neuron terhadap input dari layer sebelumnya. Implementasi backward propagation dilakukan dengan menghitung nilai error dari setiap neuron terhadap layer setelahnya, lalu dilakukan update bobot setelah melalui berbagai perhitungan.

Berdasarkan hasil pengujian yang dilakukan, terlihat bahwa setelah dilakukan analisis berbagai pengaruh, ditemukan set parameter yang cukup optimal yang setelah dilakukan perbandingan dengan library Scikit-Learn pun hasilnya tidak berbeda jauh. Dengan keberhasilan ini, FFNN from scratch yang telah dibuat dapat digunakan menjadi model Machine Learning yang lebih lanjut kedepannya dengan berbagai pengembangan.

3.2 Saran

Untuk pengembangan selanjutnya dari model Artificial Neural Network (ANN) - Feed Forward Neural Network (FFNN) from scratch yang telah dibuat adalah perbaikan bagaimana model melakukan perhitungan / komputasi di balik layar dengan lebih cepat dan efisien. Selain itu, perlu juga dilakukan analisis secara lebih mendalam bagaimana model FFNN ini mampu menghasilkan akurasi prediksi yang lebih baik dengan tetap menjaga agar tidak terjadi overfitting.

Penulis juga memberikan rekomendasi untuk mempelajari model secara lebih lanjut melalui berbagai referensi yang lebih luas sebelum melakukan pengembangan secara lebih lanjut. Dengan itu, celah untuk optimalisasi dan minimalisasi error yang mungkin terjadi juga akan semakin meningkat.

REFERENSI

[1] Asisten Lab AI 2022. (2026). Tugas Besar 1 IF3270 Pembelajaran Mesin Feed Forward Neural Network (FFNN).

https://drive.google.com/file/d/1lIRrfS_IxoxE9cDx5oTI-tseAJqiTLup/view?usp=sharing

[2] Tim Pengajar IF3270 (2026). Artificial Neural Network (ANN): Feed Feed Forward Neural Network (FFNN). [Lecture slides]. Informatika, STEI – Institut Teknologi Bandung.

<https://drive.google.com/file/d/1OgUoMqhF7WSwkqTzTw09N5eSbklTIzQE/view?usp=sharing>

LAMPIRAN

Kontribusi Tiap Anggota

Nama	NIM	Pembagian Tugas
Muhammad Raihan Nazhim Oktana	13523021	Forward Propagation & Laporan Semua Bab
Muhammad Luqman Hakim	13523044	Functions & Backward Propagation & Laporan 2.2
Faqih Muhammad Syuhada	13523057	Main Program & Visualization & Laporan 2.2

Tautan GitHub

Link: <https://github.com/FaqihMSY/FFNN-from-scratch>

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Muhammad Raihan Nazhim Oktana (13523021), Muhammad Luqman Hakim (13523044), dan Faqih Muhammad Syuhada (13523057)

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika (STEI) - Komputasi

Kode Mata Kuliah : IF3270-24

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 1 IF3270 Pembelajaran Mesin Feed Forward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1
Lainnya, Jelaskan: -	Tidak	1

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Maret 2026



Muhammad Raihan Nazhim Oktana
13523021



Muhammad Luqman Hakim
13523044



Faqih Muhammad Syuhada
13523057